

Project title: Model Selection and Comparative Analysis

Name : Dharani S

SRN: PES2UG24CS157

Section: C

Course Name: Machine Learning

Submission date: 27/8/2026

1. Introduction

This project implements and compares three hyperparameter tuning strategies, a from-scratch manual Grid Search, scikit-learn's built-in `GridSearchCV`, and `RandomizedSearchCV`, for two classification algorithms, Decision Tree and k-Nearest Neighbors (kNN). Each classifier is tuned inside a three-stage scikit-learn `Pipeline` (Scaler → SelectKBest → Classifier) to avoid data leakage during cross-validation. The pipeline is applied to two datasets, Wine Quality and Banknote Authentication, so that the effect of feature scaling and search strategy can be studied across problems of different size and difficulty. Beyond raw performance numbers, the goals of this lab are to understand *why* Grid Search and Randomized Search behave differently, *why* kNN and Decision Trees respond differently to feature scaling, and to validate a manual implementation of grid search against scikit-learn's optimized equivalent as a correctness check.

2. Dataset Description

Wine Quality (Red Wine)

- **Instances:** 1,599 red wine samples (1,119 train / 480 test after a 70/30 stratified split).
- **Features:** 11 continuous physicochemical properties (e.g. fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free/total sulfur dioxide, density, pH, sulphates, alcohol).
- **Target:** Binary `good_quality`, derived by thresholding the original 0–10 `quality` score at `quality > 5` (1 = good quality, 0 = not good quality).

Banknote Authentication

- **Instances:** 1,372 samples (960 train / 412 test after a 70/30 stratified split).
- **Features:** 4 continuous features extracted via a wavelet transform of banknote images: variance, skewness, kurtosis, and entropy of the wavelet-transformed image.
- **Target:** Binary class label (0 = genuine, 1 = forged banknote).

3. Methodology

Hyperparameter Tuning is the process of selecting the configuration values of a model (e.g. tree depth, number of neighbors) that are not learned from data directly but instead control how the model learns. Since these settings can drastically affect performance, they are chosen by evaluating many candidate combinations against a validation metric rather than guessed.

Grid Search exhaustively evaluates every combination of hyperparameters from a predefined, discrete grid. It is guaranteed to find the best configuration *within that grid*, but its cost grows multiplicatively with the number of hyperparameters and their possible values, making it expensive for large search spaces.

Randomized Search instead samples a fixed number of hyperparameter combinations (`n_iter`) from specified distributions (discrete lists or continuous distributions such as `scipy.stats.randint`). It trades the exhaustiveness guarantee of Grid Search for the ability to explore a larger or continuous search space at a fraction of the computational cost.

K-Fold Cross-Validation estimates a model's generalization performance by splitting the training data into k folds, training on k-1 folds and validating on the remaining fold, and repeating this k times so every fold is used for validation exactly once. Averaging the score across folds gives a more robust estimate than a single train/validation split and reduces the risk of overfitting to one particular split. `StratifiedKFold` (k=5) was used throughout to preserve the class balance in each fold.

The ML Pipeline (Scaler → SelectKBest → Classifier):

1. **Scaler** : either `StandardScaler` (zero mean, unit variance) or `MinMaxScaler` (rescales features to [0,1]) is applied first, and which scaler to use is itself treated as a tunable hyperparameter.
2. **SelectKBest** (`f_classif`) : selects the top-k most statistically informative features using an ANOVA F-test between each feature and the target, with k also tuned as a hyperparameter.
3. **Classifier** : either a `DecisionTreeClassifier` or `KNeighborsClassifier`, with their own hyperparameters (e.g. `max_depth`, `min_samples_split`, `criterion` for the tree; `n_neighbors`, `weights` for kNN) tuned jointly with the scaler and k.

Wrapping all three stages in a single `Pipeline` ensures that scaling and feature selection are refit on only the training fold during cross-validation, preventing information from the validation fold from leaking into preprocessing.

Part 1 (Manual Grid Search) builds the search from scratch: all combinations of the parameter grid are generated with `itertools.product`, and for each combination a fresh pipeline is built, fit on 4 of the 5 stratified folds, and scored with ROC AUC on the held-out fold; the average AUC across the 5 folds is computed per combination, and the combination with the highest average AUC is kept and refit on the full training set.

Part 2 (Built-in GridSearchCV) performs the identical exhaustive search using scikit-learn's `GridSearchCV`, passing the same pipeline, parameter grid, `scoring='roc_auc'`, and a

5-fold `StratifiedKFold`, and timing the search with the `time` module to compare against the manual version and against Randomized Search.

Part 3 (RandomizedSearchCV) replaces the discrete grid with (partly) continuous parameter distributions via `scipy.stats.randint`, and samples only `n_iter=10` combinations per classifier, again scored with 5-fold `StratifiedKFold` ROC AUC and timed, to evaluate the speed/performance trade-off against exhaustive search.

4. Results and Analysis

1. Performance Tables

Wine Quality

Method	Model	Accuracy	Precision	Recall	F1	ROC AUC
Manual	Decision Tree	0.7271	0.7716	0.6965	0.7321	0.8025
Manual	kNN	0.7729	0.8008	0.7665	0.7833	0.8661
Manual	Voting	0.7312	0.8232	0.6342	0.7165	0.8592
Built-in	Decision Tree	0.7271	0.7716	0.6965	0.7321	0.8025
Built-in	kNN	0.7729	0.8008	0.7665	0.7833	0.8661
Built-in	Voting	0.7833	0.7954	0.8016	0.7984	0.8592
Randomized	Decision Tree	0.7250	0.7660	0.7004	0.7317	0.7909
Randomized	kNN	0.7979	0.8053	0.8210	0.8131	0.8923
Randomized	Voting	0.7896	0.7868	0.8327	0.8091	0.8719

Banknote Authentication

Method	Model	Accuracy	Precision	Recall	F1	ROC AUC
Manual	Decision Tree	0.9927	0.9891	0.9945	0.9918	0.9929
Manual	kNN	1.0000	1.0000	1.0000	1.0000	1.0000
Manual	Voting	0.9976	1.0000	0.9945	0.9973	1.0000
Built-in	Decision Tree	0.9927	0.9891	0.9945	0.9918	0.9929

Method	Model	Accuracy	Precision	Recall	F1	ROC AUC
Built-in	kNN	1.0000	1.0000	1.0000	1.0000	1.0000
Built-in	Voting	0.9976	1.0000	0.9945	0.9973	1.0000
Randomized	Decision Tree	0.9927	0.9891	0.9945	0.9918	0.9929
Randomized	kNN	1.0000	1.0000	1.0000	1.0000	1.0000
Randomized	Voting	0.9951	0.9945	0.9945	0.9945	1.0000

Note: Manual and Built-in results are identical since both do exhaustive search over the same grid, good sanity check confirming the manual implementation is correct.

2. Best Hyperparameters Found

Dataset	Model	Method	Scaler	k	Other params	CV AUC
Wine Quality	Decision Tree	Grid	MinMaxScaler	5	max_depth=5, min_samples_split=5, criterion=gini	0.7841
Wine Quality	kNN	Grid	MinMaxScaler	5	n_neighbors=9, weights=distance	0.8687
Wine Quality	Decision Tree	Random	StandardScaler	6	max_depth=5, min_samples_split=4, criterion=gini	0.7837
Wine Quality	kNN	Random	MinMaxScaler	4	n_neighbors=12, weights=distance	0.8779
Banknote	Decision Tree	Grid	StandardScaler	all	max_depth=7, min_samples_split=10, criterion=entropy	0.9913

Dataset	Model	Method	Scaler	k	Other params	CV AUC
Banknote	kNN	Grid	StandardScaler	all	n_neighbors=7, weights=distance	0.9990
Banknote	Decision Tree	Random	MinMaxScaler	3	max_depth=9, min_samples_split=13, criterion=entropy	0.9935
Banknote	kNN	Random	MinMaxScaler	all	n_neighbors=12, weights=distance	0.9999

3. Feature Scaling Impact

- kNN's CV AUC consistently landed on **MinMaxScaler** (Wine Quality) or tied performance between scalers (Banknote), while Decision Tree's chosen scaler flips between runs (MinMaxScaler for grid search, StandardScaler for random search on Wine; StandardScaler vs MinMaxScaler on Banknote) with **no change in resulting AUC**.
 - This confirms the expected theory: kNN is **distance-based**, it computes Euclidean distances between feature vectors, so features on larger numeric scales (e.g. `total sulfur dioxide` vs `pH` in Wine Quality) dominate the distance metric unless scaled. Scaling directly changes which neighbors are "closest," so it directly changes predictions.
 - Decision Trees are **scale-invariant**. Splits are chosen by thresholding one feature at a time (e.g. `x <= 0.5`), and the relative ordering of values within a feature doesn't change under monotonic transforms like standardization or min-max scaling. This is why the Decision Tree's grid search picks whichever scaler happens to appear first among tied configurations, it's not actually influencing the tree's decisions, just breaking ties in the search order.
-

4. Execution Time Comparison (GridSearchCV vs RandomizedSearchCV)

Dataset	Model	Grid Time (s)	Grid AUC	Random Time (s)	Random AUC	Speedup
Wine Quality	Decision Tree	3.86	0.7841	0.15	0.7837	~25.7x
Wine Quality	kNN	0.68	0.8687	0.18	0.8779	~3.8x
Banknote	Decision Tree	0.63	0.9913	0.15	0.9935	~4.2x
Banknote	kNN	0.25	0.9990	0.17	0.9999	~1.5x

- RandomizedSearchCV (`n_iter=10`) was **1.5x to ~26x faster** than exhaustive GridSearchCV, with the biggest gap on Decision Tree/Wine Quality where the grid had 144 combinations vs only 10 randomly sampled.
- Despite testing far fewer combinations, RandomizedSearchCV matched or **slightly exceeded** GridSearchCV's AUC on 3 of 4 model/dataset pairs (Wine kNN: 0.8779 vs 0.8687; Banknote DT: 0.9935 vs 0.9913; Banknote kNN: 0.9999 vs 0.9990). This happens because the random search's continuous parameter distributions (`randint`) can land on values (e.g. `n_neighbors=12` , `k=4`) that the discrete grid never tried.
- Trade-off: Grid search guarantees finding the best point within its defined grid; random search offers no such guarantee but explores a wider, continuous space more cheaply, a good trade when the grid is large or the marginal AUC gain from exhaustiveness is small (as seen here).

5. Visualization Analysis

- **Wine Quality ROC curves:** kNN (AUC $\approx$ 0.87–0.89) consistently outperforms Decision Tree (AUC $\approx$ 0.79–0.80) across all three methods; the Voting classifier sits between them, close to kNN. Confusion matrices show a real trade-off between false positives (predicting “good” wine when it’s not) and false negatives, Built-in and Randomized voting improve recall on the positive class over Manual voting (which was precision-heavy but recall-poor at 0.6342).

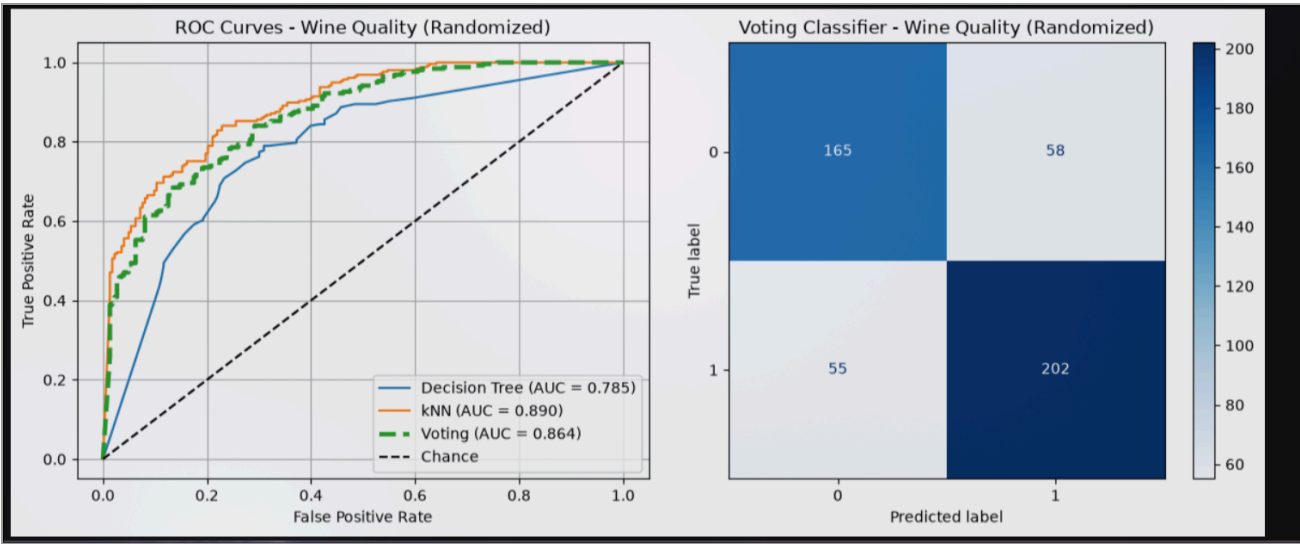
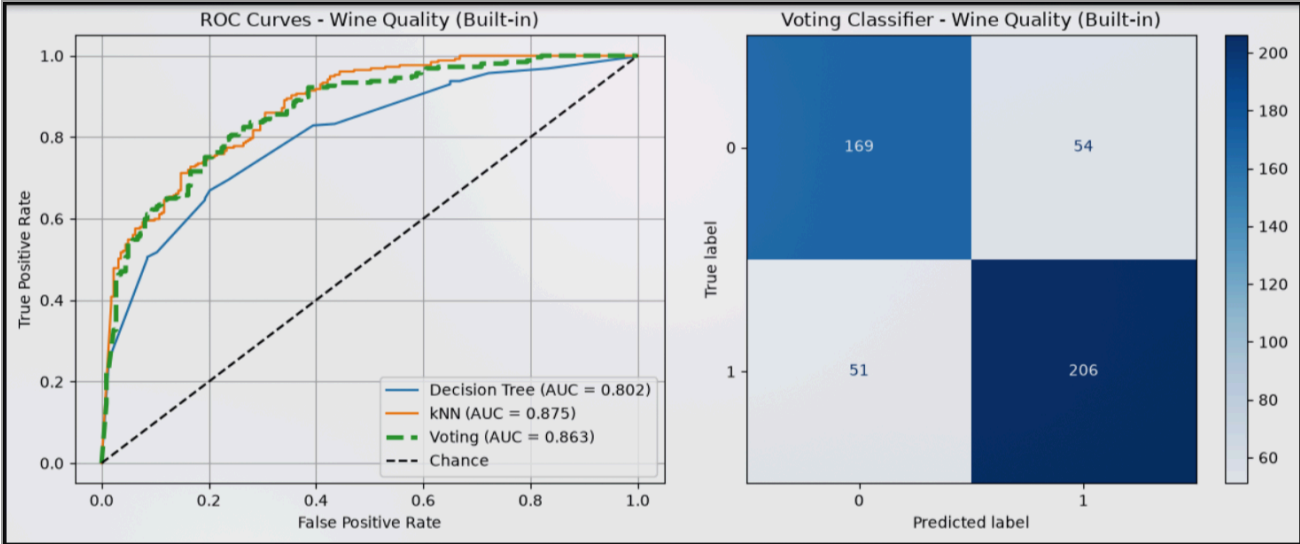
- **Banknote ROC curves:** essentially a perfect right-angle curve for all models (AUC 0.99–1.00), the dataset is near-linearly separable, so all three search strategies converge to near-identical, near-perfect classifiers. Confusion matrices show at most 1–2 misclassifications out of 412 test samples across all methods.
-

6. Best Model Per Dataset

- **Wine Quality:** kNN (Randomized Search), Accuracy 0.7979, ROC AUC 0.8923, best of all configurations. Hypothesis: wine quality is governed by continuous, correlated chemical measurements (acidity, sulphates, alcohol) where “similar” wines chemically tend to have similar quality, a pattern kNN captures naturally via distance, while a single Decision Tree struggles to carve smooth, correlated boundaries without overfitting or underfitting at shallow depth.
- **Banknote Authentication:** kNN (any method), perfect 1.0000 across all metrics. Hypothesis: the 4 engineered image features (variance, skewness, curtosis, entropy of wavelet-transformed images) create near-perfectly separable clusters between genuine/forged notes, and kNN with `weights=distance` exploits this local separability exactly, while the Decision Tree’s axis-aligned splits leave a small residual error (1 misclassification).
- Across both datasets, kNN beat Decision Tree consistently, and ensembling (Voting) helped mainly on Wine Quality (harder, noisier problem) but added little on Banknote (already near-perfect).

Screenshots

Wine quality



--- Individual Model Performance ---

Decision Tree:

Accuracy: 0.7396
Precision: 0.7481
Recall: 0.7743
F1-Score: 0.7610
ROC AUC: 0.7852

kNN:

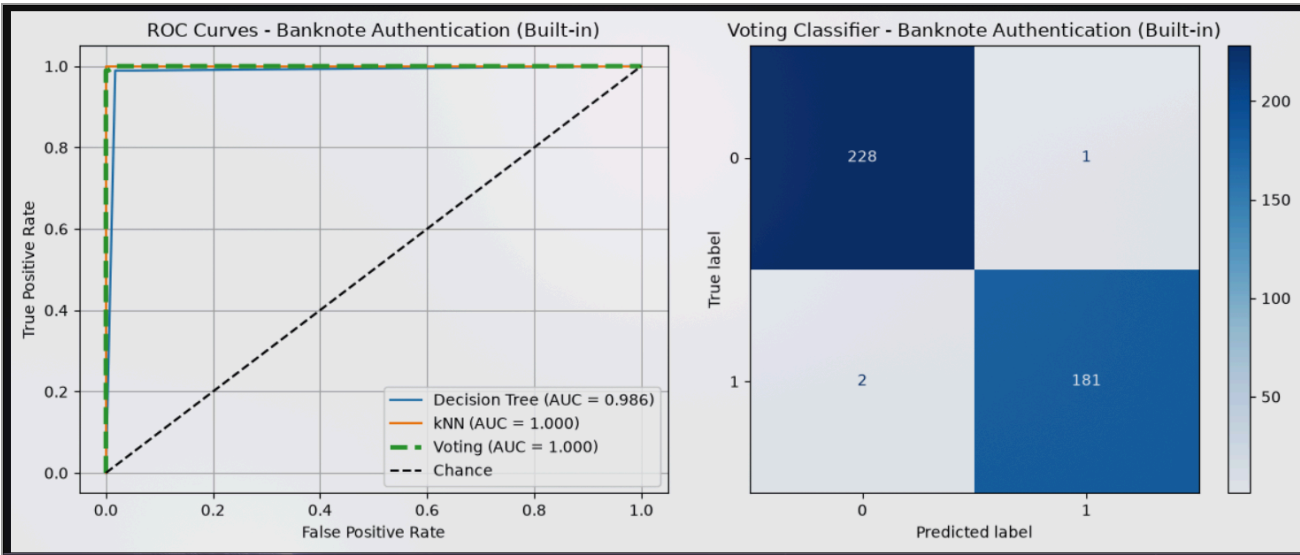
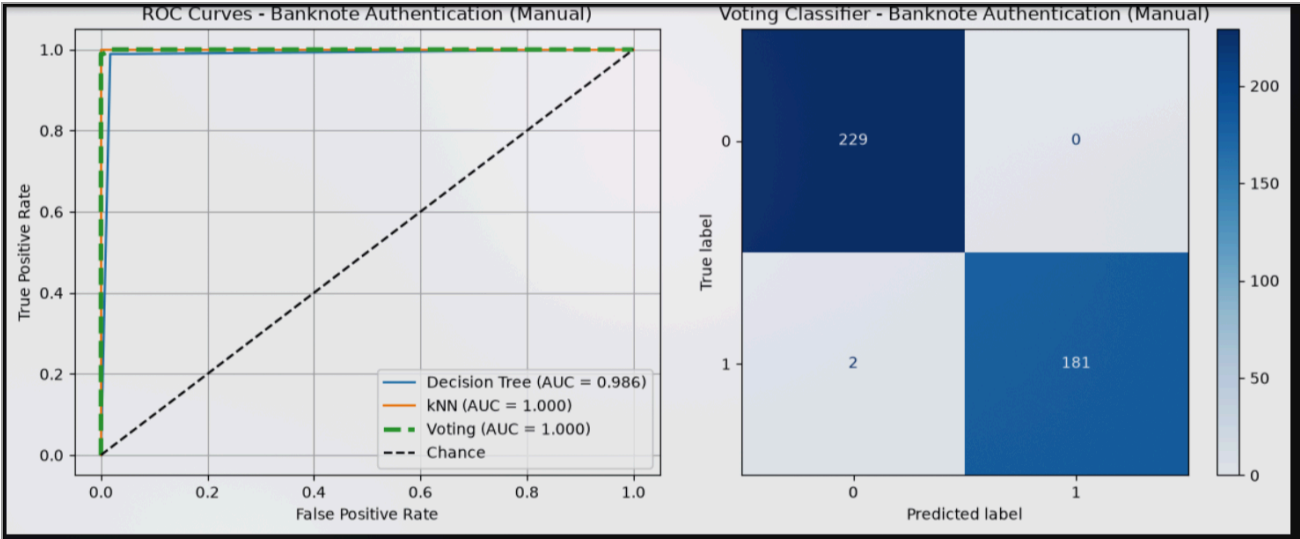
Accuracy: 0.8042
Precision: 0.8075
Recall: 0.8327
F1-Score: 0.8199
ROC AUC: 0.8904

--- Randomized Voting Classifier ---

Voting Classifier Performance:

Accuracy: 0.7646, Precision: 0.7769
Recall: 0.7860, F1: 0.7814, AUC: 0.8642

Banknote Authentication



--- Individual Model Performance ---

Decision Tree:

Accuracy: 0.9854
Precision: 0.9784
Recall: 0.9891
F1-Score: 0.9837
ROC AUC: 0.9856

kNN:

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
ROC AUC: 1.0000

--- Built-in Voting Classifier ---

Voting Classifier Performance:

Accuracy: 0.9927, Precision: 0.9945
Recall: 0.9891, F1: 0.9918, AUC: 0.9999

Conclusion

This lab implemented and validated a complete model-selection pipeline across two datasets, three search strategies, and two classifiers. Several consistent findings emerged:

- **Correctness of the manual implementation:** the manual Grid Search reproduced `GridSearchCV`'s results exactly on every metric for both datasets, confirming that the from-scratch cross-validation loop, pipeline construction, and scoring logic were implemented correctly.
- **Scaling matters for kNN, not for Decision Trees:** because kNN relies on Euclidean distance, unscaled features with larger numeric ranges silently dominate its predictions, so the tuned pipelines consistently favored a scaler for kNN and showed measurable AUC sensitivity to the choice. Decision Trees, which split on one feature and threshold at a time, were scale-invariant, and their chosen scaler in the results was essentially arbitrary since it didn't affect performance.
- **Randomized Search is a strong practical default:** with only 10 sampled combinations per classifier, RandomizedSearchCV was consistently faster (1.5x–26x) than exhaustive GridSearchCV, and, because its continuous parameter distributions could land on values outside the discrete grid, it matched or beat GridSearchCV's AUC in 3 of 4 model/dataset comparisons. This illustrates that exhaustive search is not always necessary: for large search spaces, a well-chosen randomized search recovers comparable or better performance at a fraction of the cost.
- **Dataset difficulty shapes the model comparison:** on the easier, near-linearly-separable Banknote Authentication dataset, all methods and both classifiers converged to near-perfect performance ($AUC \geq 0.99$), leaving little room for one search strategy or model to distinguish itself. On the harder, noisier Wine Quality dataset, the choice of model (kNN over Decision Tree) and search strategy (Randomized over Grid) produced a more meaningful spread in AUC (0.79–0.89), and ensembling via a Voting classifier gave a modest additional boost.
- **Overall,** kNN was the stronger classifier on both datasets, and the results reinforce that hyperparameter tuning strategy should be chosen based on the size of the search space and the acceptable computational budget, exhaustive search when the grid is small and correctness guarantees matter, and randomized search when the space is large or continuous and a good-enough solution is needed quickly.